

# Ch 02: Operating System Organization

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science  
East Texas A&M University

Summer 2026



**EAST TEXAS A&M**

- 1 Isolation Abstractions
- 2 Kernel Organization
- 3 Process Abstraction



# Ch 02: Isolation Abstractions

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science  
East Texas A&M University

Summer 2026



**EAST TEXAS A&M**

# Operating System Requirements

An operating system must fulfill three requirements:

- ① **Multiplexing:** The operating system must **time-share** resources such as CPUs and memory among processes
- ② **Isolation:** The operating system must arrange for isolation between processes
  - both because bugs are unavoidable
  - but we also cannot assume good intentions for all processes we may want to run (security)
- ③ **Interaction:** It should be possible for processes to intentionally interact
  - pipelines
  - shared memory
  - some problems are more easily solved and have better performance using parallel solutions



# An Aside on xv6 and RISC-V

- xv6 runs on a **multi-core** RISC-V microprocessor
- Written in “LP64” C (L)ong (P)ointers are 64 bits (64-bit architecture)
  - C “standard library” implementation is xv6 specific
- Will introduce some RISC-V specific architecture ideas as needed
  - [RISC-V ISA Unprivileged Architecture](#) (Waterman & Asanović, 2025a)
  - [RISC-V ISA Privileged Architecture](#) (Waterman & Asanović, 2025b)
  - [The RISC-V Reader: An Open Architecture Atlas](#) (Patterson & Waterman, 2017)
- xv6 written specifically for **qemu** emulator
  - uses “-machine virt” flag



# Abstracting Physical Resources

- Unix applications interact with storage only through the file system's `open()`, `read()`, `write()` and `close()` system calls
  - Provides some high-level abstractions like path names
  - Even if isolation is not considered, the convenience and simplicity of this abstraction is desirable
- Unix transparently switches hardware CPUs among processes
  - applications are not aware of time-sharing
- `exec()` system call abstracts memory access
  - OS controls memory (MMU)
  - OS builds memory image, decides where to map process into memory
  - OS controls process virtual address space to actual physical address space mapping
- File descriptors provide a high-level abstraction for process interaction



# User Mode, Supervisor Mode and System Calls

## Supervisor Mode

In supervisor mode the CPU is allowed to execute **privileged instructions**. For example enabling and disabling interrupts, reading and writing privileged registers that hold the address of the page table, etc.

## User Mode

Application processes execute only in user mode. They are restricted to using unprivileged user mode instructions.

- Strong isolation requires hard boundary between applications and the OS
- To achieve, the OS must arrange that processes cannot modify OS data structures or instructions
  - and cannot access other processes' memory
- CPUs provide hardware support for strong isolation
  - RISC-V has 3 privilege modes
    - ① **machine mode**: has full privilege and is mostly used only during boot
    - ② **supervisor mode**: execute **privileged instructions**, **kernel mode** or **kernel space**
    - ③ **user mode**: unprivileged instructions only, must invoke system calls to access privileged resources
- RISC-V provides **ecall** instruction for system calls, it switches CPU from user to supervisor mode.



EAST TEXAS A&M

# Summary

- There are 3 requirements an OS must fulfill 1) multiplexing, 2) isolation 3) interaction
- In addition, OS goal is to provide **abstractions**, high-level interfaces for using complicated hardware
  - All storage and device interactions through file descriptor abstraction
  - Process abstraction provides multiplexing and illusion of complete control of a cpu
  - `exec()` build virtual memory abstraction, illusion of an address space that no other process has access to
  - The unix system-call interface provides both convenience and strong isolation
- Strong isolation and protected memory require hardware support
  - All cpu architectures for general purpose OS, like the RISC-V architecture, provide supervisor and user modes (ecall instruction)





# Ch 02: Kernel Organization

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science  
East Texas A&M University

Summer 2026



**EAST TEXAS A&M**

# Kernel Organization (1 of 2)

## Monolithic Kernel

The entire operating system resides in the kernel, so that the implementations of all system calls run in supervisor mode.

### Pros:

- Don't have to divide code into parts that do not require supervisor privileges
- Easy for different parts of OS to cooperate

### Cons:

- Tend to grow large and complex

## Microkernel

A microkernel aims to put an absolute minimum of functionality in the kernel itself, so that little code executes in supervisor mode, and so that the kernel is easy to understand and analyze for correctness.

### Pros:

- In theory easier to understand kernel code, less complex

### Cons:

- Harder to design
- can be inefficient compared to monolithic approach

# Kernel Organization (2 of 2)

Irregardless, micro and monolithic kernel designs all implement:

- system calls
- page tables (virtual memory management)
- process abstraction
- concurrency mechanisms
- file system abstractions

Which will be focus of topics in the course. Xv6, like the Unix v6 it replicates, is a monolithic kernel design. But it is a small teaching OS, so it is actually much smaller than many modern microkernels in production.



# Code: xv6 Organization

| area      | File          | Description                                           |
|-----------|---------------|-------------------------------------------------------|
| Boot      | entry.S       | Very first boot instructions                          |
|           | main.c        | Control initialization of other modules               |
|           | start.c       | Early machine-mode boot code                          |
| Processes | exec.c        | exec() system call                                    |
|           | proc.c        | Processes and scheduling                              |
|           | swtch.S       | Thread switching                                      |
|           | sysproc.c     | Process-related system calls                          |
| Traps     | kernelvec.S   | Handle traps from kernel code                         |
|           | trampoline.S  | Handle traps from user code                           |
|           | trap.c        | C code to handle and return from traps and interrupts |
|           | syscall.c     | Dispatch system calls to handling function            |
| Memory    | vm.c          | Manage page tables and address spaces                 |
|           | kalloc.c      | Physical page allocator                               |
| Devices   | console.c     | Connect to the user keyboard and screen               |
|           | plic.c        | RISC-V interrupt controller                           |
|           | printf.c      | Formatted output to the console                       |
|           | uart.c        | Serial-port console device driver                     |
|           | virtio_disk.c | Disk device driver                                    |
|           | bio.c         | Disk block cache for the file system                  |
|           | file.c        | File descriptor support                               |
| FS        | fs.c          | File system                                           |
|           | log.c         | File system logging and crash recovery                |
|           | sysfile.c     | File-related system calls                             |
|           | pipe.c        | Pipes                                                 |
|           | sleeplock.c   | Locks that yield the CPU                              |
| Misc      | spinlock.c    | Locks that don't yield the CPU                        |
|           | string.c      | C string and byte-array library                       |



# Summary

- Generally OS can currently be divided into 2 general design approaches
  - **monolithic kernel** traditional approach, entire OS resides in kernel
  - **microkernel** put absolute minimum of code in kernel that is needed, pull services where possible out to user mode processes
- OS designs all implement key design elements
  - system calls (API / abstraction)
  - page tables (virtual memory management)
  - concurrency mechanisms (processes and abstractions for multiplexing)
  - file system abstractions



# Ch 02: Process Abstraction

## xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science  
East Texas A&M University

Summer 2026



**EAST TEXAS A&M**

# Process Overview

- The unit of isolation in xv6 (as in other Unix and most all OS) is a **process**
  - process is an abstraction
  - prevents one process from reading or writing another's memory, CPU or file descriptors
  - also prevents process from wrecking the kernel itself
    - so that processes cannot subvert the kernel's isolation mechanisms
- Mechanisms used by kernel to implement processes
  - user/supervisor mode flag
  - address spaces (through page tables)
  - time-slicing of threads
- Process abstraction provides illusion to a program that it has its own private machine
  - Provides program with what appears to be a private memory system **virtual address space**
  - Also provides program with what appears its own CPU to execute program instructions with



# Process Overview (1 of 2)

- Process abstraction provides illusion to a program that it has its own private machine
  - Uses page tables (which are implemented by hardware)
    - A page table maps a **virtual address** to a **physical address**
    - Maintains a separate page table for each process
- Address space includes
  - **user memory** starting at virtual address 0
  - instructions (text sections, read only)
  - data (data sections, read only)
  - stack (function call stack, read/write)
  - heap (dynamically allocated memory, malloc and free, read/write)
- Though pointers are 64 bit wide, xv6 uses only 38 bits to look up in page table
  - Thus  $MAXVA = 2^{38} - 1 = 0x3fffffff$

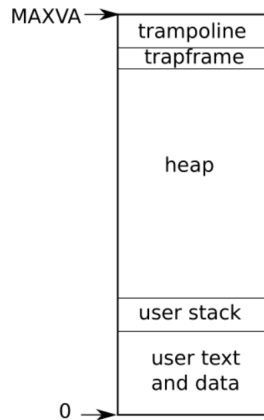


Figure 1: Layout of a process's virtual address space (Cox et al. 2026, pg.26)





## Process Overview (2 of 2)

- xv6 uses 12 bit page sizes, so  $\text{PAGESIZE} = 2^{12} = 4096$
- Top two pages of user space are dual mapped into kernel space and these two pages are used to transition into and out of kernel space
- **trampoline page** (4096 bytes)
  - contains the code to transition in and out of the kernel
- **trapframe page** (4096 bytes)
  - where kernel saves the process's user registers
- xv6 kernel maintains many pieces of state in struct `proc`
  - see the `kernel/proc.h` for declaration of this structure
  - page table
  - kernel stack
  - run state

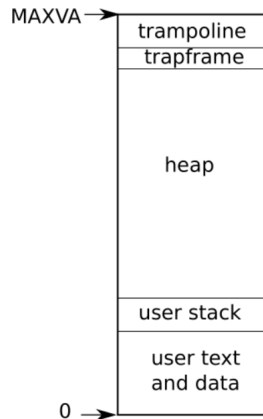


Figure 2: Layout of a process's virtual address space (Cox et al. 2026, pg.26)



# Process Switching

- To switch a CPU between processes, the kernel suspends the thread currently running and saves its state, and restores the state of another process.
- Much of the state of a thread (local variables, function call return address) is stored on the thread's stacks
- Each process has two stacks
  - ① **user stack**: For when processing and calling user instructions
  - ② **kernel stack**: (`p->kstack`) when process enters kernel mode
- Process makes a system call by executing the RISC-V `ecall` instruction
  - Switches to supervisor mode
  - Changes program counter to a kernel-defined entry point
  - Returns to user space by executing `sret` instruction
    - switches back to user mode
    - resume executing user instructions just after the `ecall` system call instruction
    - A processes thread can “block” in kernel to wait for I/O, and resume when I/O finishes



- The struct `proc p->state` indicates (see enum `procstate`):
  - free process table entry `UNUSED`
  - allocated `USED`
  - ready to run `RUNNABLE`
  - currently running `RUNNING`
  - waiting for I/O `SLEEPING`
  - exiting `ZOMBIE`



# Process Pagetable

- The struct `proc p->pagetable` holds the process's page table
  - xv6 causes the paging hardware to use process's `p->pagetable` when executing that process in user space
  - A process's page table also is record of address of physical pages allocated to store a process's memory



# Code: Starting xv6, the first process and system calls

- ① Hardware boot loader copies xv6 kernel into memory at 0x80000000
- ② Boot loader jumps to `_entry` (`kernel/entry.S`)
  - paging hardware disabled at this point
  - sets up a stack so that xv6 can run C code
  - loads stack pointer register `sp` (actually separate stack for each cpu)
  - calls `start` which now starts using stack for calls
- ③ Function `start` (`kernel/start.c`)
  - still in machine mode from boot loader at this point
  - program clock chip to generate timer interrupts
  - switch to supervisor mode and jump to `main` (last 2 expressions)
- ④ Function `main` (`kernel/main.c`)
  - initializes a bunch of things, only on `cpu0` if multi-cpu
  - ending with `userinit()` to create the first process
- ⑤ After calling `kexec`, `forkret()` returns to user space in the `/init` process
  - creates a new console device
  - opens it as file descriptors 0, 1 and 2
  - starts shell on console



# Summary

- The unit of isolation in unix-like OS including xv6 is the **process** abstraction
- Uses user/supervisor mode flags, page table virtual address spaces and time slicing to implement process abstraction and isolation
- Process address space loads user code and data, has a stack and heap
  - Special trampoline and trapframes mapped to both user and kernel space
    - **trampoline page** code used to transition in and out of kernel mode
    - **trapframe page** saves process register state to restore
  - `struct proc` in `kernel/proc.h` are process table entries that kernel uses to maintain process state
- Process switching is done by kernel, suspends currently running process, saves its state, and restores state of another process to run
  - process state in process table is running, runnable, sleeping, zombie, etc.
  - process `struct pagetable` holds the process's page table



# Security Model

- OS must assume that a process's user-level code will do its best to wreck the kernel or other processes
  - Zero-trust model
- Kernel's goals
  - restrict each process so that it can only access its own user memory
  - can only use the 32 general-purpose RISC-V registers
  - can only affect kernel and other processes in the way that system calls are intended
- Kernel code is trusted, assumed to
  - well-meaning and written by careful programmers
  - bug-free (as possible)
  - contain nothing malicious



# Summary





# References

- Cox, R., Kaashoek, F., & Morris, R. (2026). *Xv6: A simple, unix-like teaching operating system*. Retrieved September 2, 2025, from <https://github.com/mit-pdos/xv6-riscv-book>
- Patterson, D. A., & Waterman, A. (2017). *The RISC-V reader: An open architecture atlas*. Strawberry Canyon LLC.
- Waterman, A., & Asanović, K. (Eds.). (2025a). *The RISC-V instruction set manual volume I: Unprivileged architecture*. Retrieved May 8, 2025, from <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj7Omv8tFtkp/view>
- Waterman, A., & Asanović, K. (Eds.). (2025b). *The RISC-V instruction set manual volume II: Privileged architecture*. Retrieved May 8, 2025, from [https://drive.google.com/file/d/17GeetSnT5wW3xNuAHI95-SI1gPGd5sJ\\_/view](https://drive.google.com/file/d/17GeetSnT5wW3xNuAHI95-SI1gPGd5sJ_/view)

